AI/ML DEEP DIVE

Deep Learning & Neural Networks

From a single neuron to transformers — the intuition, the math that matters, idiomatic PyTorch, and interview-ready answers for modern deep learning.

deep-learning · neural-networks · cnn · rnn · transformers · backprop · interview-prep

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. 1. What it is and why it matters
2. 2. Setup & getting started
3. 3. The neuron, layers & activations
4. 4. Forward pass, loss & backpropagation
5. 5. Optimizers & the training loop
6. 6. Convolutional networks (vision)
7. 7. Sequence models: RNN, LSTM & GRU
8. 8. Attention & transformers
9. 9. Embeddings & representation
10. 10. Regularization & normalization
11. 11. Transfer learning & fine-tuning
12. 12. A worked mini-example: end-to-end training
13. 13. Cheat-sheet & comparisons
14. 14. Best practices & common pitfalls
15. 15. Interview Questions & Answers
16. 16. Further learning

Deep learning is the practice of composing many layers of simple, differentiable building blocks into functions rich enough to learn images, language, audio and more directly from raw data — trained end-to-end by gradient descent. This guide takes you from a single neuron to transformers, with the intuition, the math that matters, idiomatic code, and the interview answers.

> **Who this is for**
>
> An engineer who can code and wants a working, interview-ready mental model of neural networks — not a proofs course. We favor intuition plus runnable PyTorch, then show where each idea is used in production.

1. What it is and why it matters

A neural network is a **parameterized function** `y = f(x; θ)` built by stacking layers. Each layer applies a linear transform (a matrix multiply plus bias) followed by a nonlinear *activation*. Stack enough of them and the network becomes a **universal function approximator**: given enough capacity and data, it can approximate essentially any continuous mapping from inputs to outputs.

What makes it "deep" is the composition of many layers, which lets the model build a **hierarchy of features** — edges to textures to object parts to objects in vision; characters to words to phrases to meaning in language. Crucially, these features are *learned*, not hand-engineered. That is the central shift from classical machine learning.

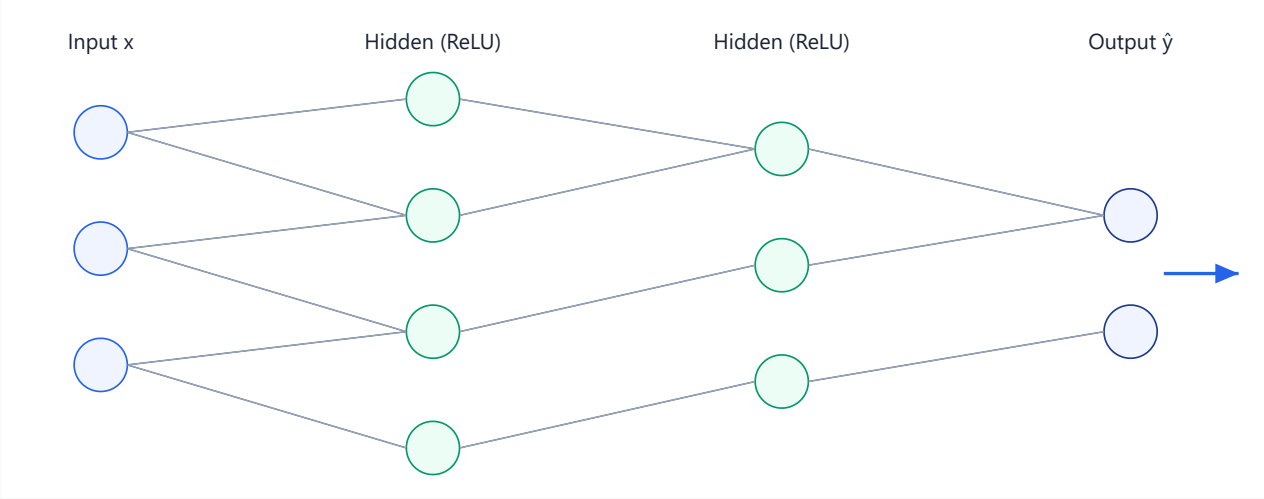


Fig 1. A feed-forward network: data flows left to right; each edge is a learnable weight, each neuron sums its inputs and applies a nonlinearity.

REPRESENTATION LEARNING

Features emerge from data instead of being hand-crafted. One architecture, many domains.

SCALES WITH DATA & COMPUTE

Performance keeps improving with more parameters, data, and GPU hours — the basis of modern "foundation models".

END-TO-END DIFFERENTIABLE

Every piece is differentiable, so the whole pipeline is optimized jointly by backprop.

2. Setup & getting started

PyTorch is the de-facto research and production framework in 2026; JAX and TensorFlow/Keras remain common. We use PyTorch throughout.

BASH

```bash
# Create an isolated environment
python -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\activate

# Install PyTorch (CPU build shown; see pytorch.org for the exact CUDA command)
pip install torch torchvision torchaudio

# Common companions
pip install numpy matplotlib scikit-learn tensorboard
```

PYTHON

```python
import torch

# Pick the best available accelerator (CUDA, Apple MPS, else CPU)
device = (
    "cuda" if torch.cuda.is_available()
    else "mps" if torch.backends.mps.is_available()
    else "cpu"
)
print("Using", device, "| torch", torch.__version__)

x = torch.randn(3, 4, device=device)   # a 3x4 tensor on the device
print(x.shape, x.dtype)                 # torch.Size([3, 4]) torch.float32
```

✦ **Tensors**

A tensor is just an n-dimensional array that tracks gradients and lives on a device. Scalars are 0-D, vectors 1-D, matrices 2-D, a batch of RGB images is 4-D: `[N, C, H, W]`.

3. The neuron, layers & activations

A single neuron computes a weighted sum of its inputs plus a bias, then applies an activation `a = ϕ(w·x + b)`. A **layer** is many neurons sharing the same inputs, which vectorizes to a matrix multiply: `z = Wx + b`, `a = ϕ(z)`.

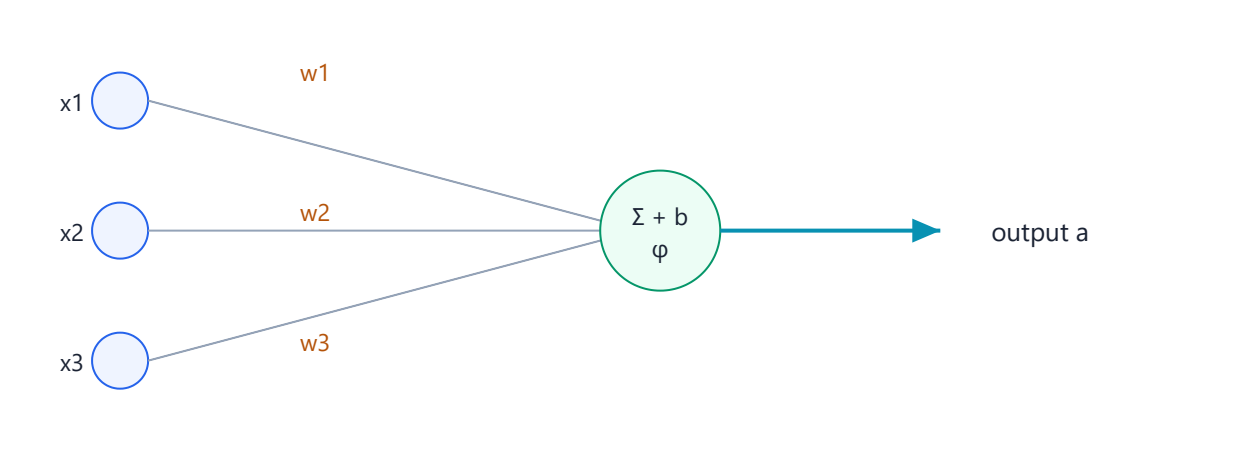


Fig 2. One neuron: weighted sum of inputs plus bias, passed through activation φ.

Why activations must be nonlinear

Without a nonlinearity, stacking layers collapses: a composition of linear maps is itself a single linear map, so depth buys nothing. The activation is what gives the network the ability to bend decision boundaries and model complex functions.

Activation	Formula	Range	Use when	Watch out for
ReLU	max(0, z)	[0, ∞)	Default hidden-layer choice	"Dead" neurons stuck at 0
Leaky ReLU / GELU	small slope < 0 / smooth	(-∞, ∞)	Avoid dead units; GELU in transformers	Slightly more compute
Sigmoid	$1 / (1 + e^{-z})$	(0, 1)	Binary output probability	Saturates → vanishing gradient
Tanh	$(e^{z}-e^{-z})/(e^{z}+e^{-z})$	(-1, 1)	Zero-centered hidden units (RNNs)	Also saturates
Softmax	$e^{z_i} / \Sigma e^{z_j}$	(0, 1), sums to 1	Multi-class output layer	Not for hidden layers

PYTHON

```python
import torch.nn as nn

# A small multilayer perceptron (MLP) for 28x28 images → 10 classes
mlp = nn.Sequential(
    nn.Flatten(),            # [N,1,28,28] → [N,784]
    nn.Linear(784, 256),     # learnable W (256x784) + b (256)
    nn.ReLU(),               # nonlinearity
    nn.Dropout(0.2),         # regularization (see section 8)
    nn.Linear(256, 10),      # logits for 10 classes (no softmax here)
)
# nn.CrossEntropyLoss applies log-softmax internally, so output raw logits.
```

4. Forward pass, loss & backpropagation

Training repeats a four-step loop. The **forward pass** computes predictions and a scalar **loss**. **Backpropagation** computes the gradient of the loss with respect to every parameter by applying the

chain rule layer by layer, from output back to input. The **optimizer** then nudges each parameter in the direction that reduces the loss.



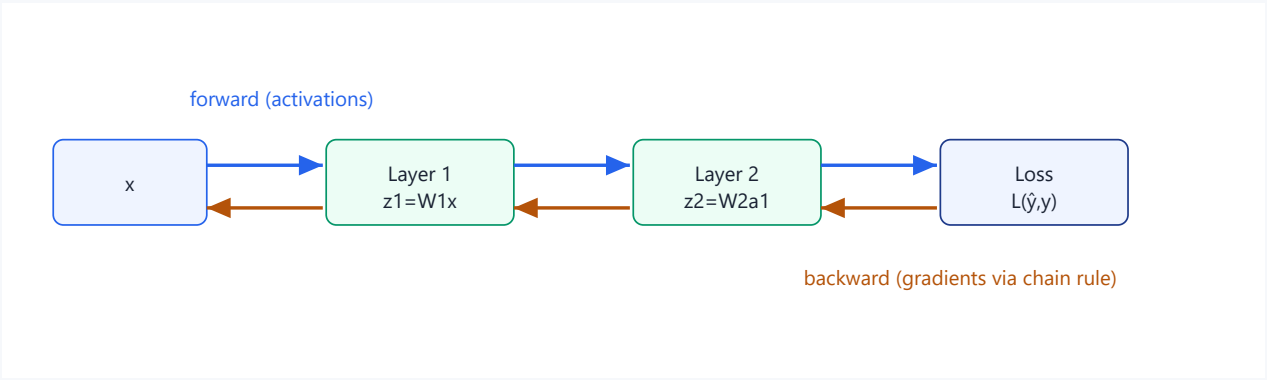


Fig 3. Backprop reuses cached forward activations to compute ∂L/∂W at each layer, moving right to left.

Autograd makes this automatic: every tensor op is recorded on a graph, and `loss.backward()` walks it in reverse.

PYTHON

```python
import torch

# Manual illustration of autograd on a tiny computation
w = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(1.0, requires_grad=True)
x = torch.tensor(3.0)

y_hat = w * x + b          # forward: 7.0
loss = (y_hat - 10.0) ** 2  # squared error vs target 10  → 9.0
loss.backward()            # backward: fills .grad

# dL/dw = 2*(y_hat-10)*x = 2*(-3)*3 = -18 ;  dL/db = 2*(-3) = -6
print(w.grad, b.grad)      # tensor(-18.) tensor(-6.)
```

⚠ **Vanishing & exploding gradients**

In deep stacks, repeated multiplication of small (<1) derivatives shrinks gradients toward zero; large ones blow up. Cures: ReLU/GELU activations, residual connections, normalization, careful initialization (He/Xavier), and gradient clipping.

5. Optimizers & the training loop

Gradient descent updates parameters as `θ ← θ − η · ∂L/∂θ`, where `η` is the learning rate. We rarely use the full dataset per step; **mini-batch SGD** estimates the gradient from a small batch, trading a little noise for huge speedups and better generalization.

Optimizer	Idea	Pros	Typical use
SGD	Plain gradient step	Simple, strong generalization	Vision with schedules
SGD + Momentum	Accumulate a velocity	Dampens oscillation, faster	CNNs, ResNets
RMSProp	Per-param adaptive scaling	Handles non-stationary objectives	RNNs
Adam / AdamW	Momentum + adaptive scaling	Robust default, fast convergence	Transformers, most models

✦ **AdamW is the safe default**
AdamW decouples weight decay from the gradient update and is the standard choice for transformers and most modern training. Pair it with a learning-rate warmup then cosine decay.

PYTHON

```python
import torch, torch.nn as nn
from torch.utils.data import DataLoader

def train_one_epoch(model, loader: DataLoader, optimizer, device):
    model.train()                                   # enable dropout/batchnorm training
    loss_fn = nn.CrossEntropyLoss()
    running = 0.0
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()                       # clear old gradients
        logits = model(xb)                          # forward
        loss = loss_fn(logits, yb)                  # scalar loss
        loss.backward()                             # backprop: fill .grad
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)  # guard against explosions
        optimizer.step()                            # update parameters
        running += loss.item() * xb.size(0)
    return running / len(loader.dataset)

# optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=0.01)
```

The #1 beginner bug

Forgetting `optimizer.zero_grad()`. PyTorch *accumulates* gradients by default, so without zeroing, each step uses the sum of all previous gradients and training diverges or stalls.

6. Convolutional networks (vision)

Fully-connected layers ignore spatial structure and explode in parameters on images. A **convolutional layer** slides a small learnable filter across the image, computing the same weighted sum everywhere. This gives two superpowers: **parameter sharing** (one filter, whole image) and **translation equivariance** (a cat is a cat wherever it appears).

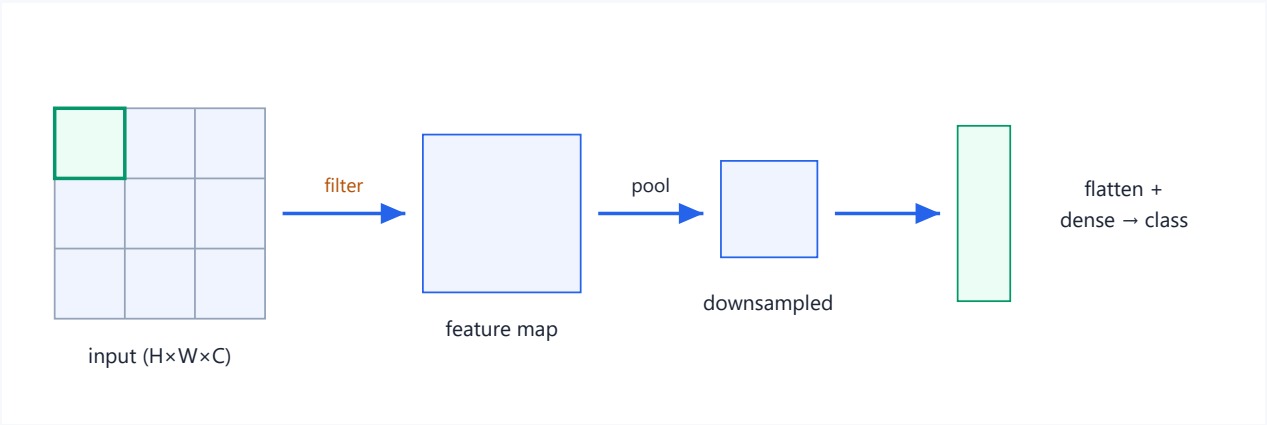


Fig 4. A CNN: stacks of conv + pooling shrink spatial size while growing channel depth, then dense layers classify.

PYTHON

```python
import torch.nn as nn

class SmallCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),  # [N,32,28,28]
            nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2),                             # [N,32,14,14]
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2),                             # [N,64,7,7]
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 7 * 7, 128), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(128, num_classes),
        )

    def forward(self, x):
        return self.classifier(self.features(x))
```

Key hyperparameters of a conv layer: **kernel size** (receptive field), **stride** (step size / downsampling), **padding** (preserve spatial size), and **channels** (number of filters = features learned). Landmark architectures to know: LeNet, AlexNet, VGG, **ResNet** (residual connections enabling very deep nets), and EfficientNet. Vision Transformers (ViT) now rival or beat CNNs at scale.

7. Sequence models: RNN, LSTM & GRU

For ordered data (text, time series, audio) a **recurrent network** processes one element at a time, carrying a **hidden state** that summarizes everything seen so far: `h_t = ϕ(W_x x_t + W_h h_{t-1} + b)`. The same weights are reused at every timestep.

Plain RNNs struggle with long-range dependencies because gradients vanish over many steps. **LSTMs** and **GRUs** add gating — learnable valves that decide what to keep, forget, and output — letting a cell state carry information across long sequences.

FORGET GATE

Decides how much of the previous cell state to discard.

INPUT GATE

Decides what new information to write into the cell state.

OUTPUT GATE

Decides what to expose from the cell state as the hidden output.

PYTHON

```python
import torch, torch.nn as nn

class TextClassifier(nn.Module):
    def __init__(self, vocab_size, emb=128, hidden=256, classes=2):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, emb)          # token id → vector
        self.lstm = nn.LSTM(emb, hidden, batch_first=True,  # [N, T, emb] → ...
                            bidirectional=True)
        self.head = nn.Linear(hidden * 2, classes)          # *2 for bidirectional

    def forward(self, x):                  # x: [N, T] of token ids
        e = self.embed(x)                  # [N, T, emb]
        out, (h, c) = self.lstm(e)         # out: [N, T, 2*hidden]
        pooled = out.mean(dim=1)           # average over time
        return self.head(pooled)
```

> **The shift to attention**
> RNNs are inherently sequential (step t needs step t-1), limiting parallelism and still fading over very long contexts. Transformers removed recurrence entirely and now dominate NLP, vision, audio, and multimodal work.

8. Attention & transformers

The core idea of **attention**: for each position, compute a weighted blend of all positions, where the weights say "how relevant is each other token to me right now?" Each token emits a **query**, a **key**, and a **value**. The query is matched against all keys (dot product) to produce attention weights, which blend the values.

Scaled dot-product attention: `Attention(Q,K,V) = softmax(QKᵀ / √d_k) · V`. The `√d_k` scaling keeps the dot products from growing large and saturating the softmax.

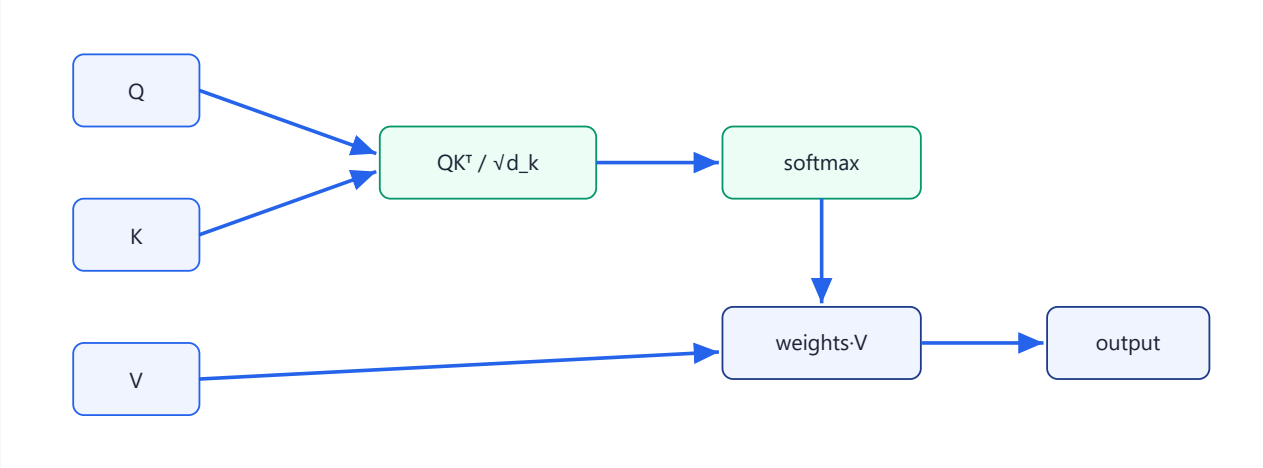


Fig 5. Scaled dot-product attention. Multi-head attention runs several of these in parallel on different learned projections.

A **transformer block** stacks: multi-head self-attention, a position-wise feed-forward network, residual connections around each, and layer normalization. Because attention is order-agnostic, **positional encodings** (sinusoidal or learned, now often rotary/RoPE) inject sequence position. Stacking these blocks gives BERT (encoder), GPT-family (decoder), and T5 (encoder-decoder).

PYTHON

```python
import torch, torch.nn as nn, torch.nn.functional as F

def scaled_dot_product_attention(q, k, v, mask=None):
    # q,k,v: [batch, heads, seq, d_k]
    d_k = q.size(-1)
    scores = (q @ k.transpose(-2, -1)) / d_k ** 0.5   # [.., seq, seq]
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float("-inf"))
    weights = F.softmax(scores, dim=-1)               # attention distribution
    return weights @ v                                # blended values

# In practice, prefer the fused, memory-efficient built-in:
# out = F.scaled_dot_product_attention(q, k, v, is_causal=True)
# or nn.TransformerEncoderLayer / nn.MultiheadAttention.
```

✦ **Why transformers won**

Self-attention relates any two positions in O(1) sequential steps and is massively parallel on GPUs. Combined with huge datasets, this is what makes modern LLMs and foundation models possible.

9. Embeddings & representation

Neural nets need numeric vectors. An **embedding** is a learned, dense, low-dimensional vector for a discrete item (a word, product, user, or category). Items that behave similarly end up close together in the vector space, so geometric distance becomes semantic similarity — the backbone of search, recommendation, and retrieval-augmented generation (RAG).

PYTHON

```python
import torch, torch.nn as nn

emb = nn.Embedding(num_embeddings=10_000, embedding_dim=64)  # 10k tokens → 64-D
ids = torch.tensor([[1, 42, 7]])        # a batch of one sequence of 3 tokens
vecs = emb(ids)                          # [1, 3, 64] learned vectors

# Cosine similarity between two learned vectors (semantic closeness)
a, b = vecs[0, 0], vecs[0, 1]
cos = torch.cosine_similarity(a, b, dim=0)
print(cos.item())
```

10. Regularization & normalization

Deep nets have enough capacity to **memorize** training data (overfit). Regularization closes the gap between training and validation performance.

DROPOUT

Randomly zero a fraction of activations each step during training, forcing redundancy. Disabled at inference. Typical p = 0.1–0.5.

WEIGHT DECAY (L2)

Penalize large weights, encouraging simpler functions. In AdamW it is decoupled from the gradient step.

BATCH NORMALIZATION

Normalize each layer's activations per mini-batch, stabilizing and speeding training. Standard in CNNs.

LAYER NORMALIZATION

Normalize across features per sample (batch-size independent). The norm of choice in transformers and RNNs.

EARLY STOPPING

Stop when validation loss stops improving; keep the best checkpoint.

DATA AUGMENTATION

Flips, crops, color jitter, mixup, cutout — cheap "new" data that improves robustness.

⚠ **BatchNorm & small batches**

BatchNorm statistics are noisy with tiny batches and leak across examples in some settings. For small batches, long sequences, or online inference, prefer LayerNorm or GroupNorm. Always call `model.eval()` at inference so BatchNorm uses running statistics and Dropout is turned off.

11. Transfer learning & fine-tuning

Training from scratch needs lots of data and compute. **Transfer learning** reuses a model pretrained on a large corpus and adapts it to your task. This is the dominant workflow in 2026: start from a foundation model, then fine-tune.

Pretrained backbone -> freeze or adapt -> new head -> fine-tune on your data

PYTHON

```python
import torch.nn as nn
from torchvision import models

# Start from an ImageNet-pretrained ResNet, retarget to 5 classes
net = models.resnet50(weights=models.ResNet50_Weights.DEFAULT)
for p in net.parameters():
    p.requires_grad = False              # freeze the backbone

net.fc = nn.Linear(net.fc.in_features, 5)  # new trainable head
# Train only net.fc first; optionally unfreeze top blocks later with a small LR.
```

✦ **PEFT & LoRA**

For large language models, full fine-tuning is expensive. Parameter-efficient methods like **LoRA** train tiny low-rank adapter matrices while freezing the base weights — a fraction of the cost and memory, with near-full-fine-tune quality.

12. A worked mini-example: end-to-end training

This puts the loop together: data, model, optimizer, train/validate, and the critical `eval()` / `no_grad()` discipline.

PYTHON

```python
import torch, torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

device = "cuda" if torch.cuda.is_available() else "cpu"

# Synthetic 2-class problem: 20 features
X = torch.randn(2000, 20)
y = (X[:, :5].sum(dim=1) > 0).long()      # label depends on first 5 features
train_ds = TensorDataset(X[:1600], y[:1600])
val_ds   = TensorDataset(X[1600:], y[1600:])
train_dl = DataLoader(train_ds, batch_size=64, shuffle=True)
val_dl   = DataLoader(val_ds, batch_size=256)

model = nn.Sequential(
    nn.Linear(20, 64), nn.ReLU(), nn.Dropout(0.2),
    nn.Linear(64, 2),
).to(device)

opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-2)
loss_fn = nn.CrossEntropyLoss()

@torch.no_grad()                          # no gradients during evaluation
def accuracy(dl):
    model.eval()
    correct = total = 0
    for xb, yb in dl:
        xb, yb = xb.to(device), yb.to(device)
        preds = model(xb).argmax(dim=1)
        correct += (preds == yb).sum().item()
        total += yb.size(0)
    return correct / total

for epoch in range(10):
    model.train()
    for xb, yb in train_dl:
        xb, yb = xb.to(device), yb.to(device)
        opt.zero_grad()
        loss = loss_fn(model(xb), yb)
        loss.backward()
        opt.step()
    print(f"epoch {epoch}: val acc = {accuracy(val_dl):.3f}")
```

13. Cheat-sheet & comparisons

Task	Output layer	Loss function
Binary classification	1 logit (+ sigmoid)	BCEWithLogitsLoss
Multi-class (single label)	C logits	CrossEntropyLoss
Multi-label	C logits (+ sigmoid)	BCEWithLogitsLoss
Regression	linear	MSELoss / L1Loss / HuberLoss

Architecture	Best for	Key mechanism	Weakness
MLP	Tabular, small vectors	Dense layers	Ignores structure; parameter-heavy
CNN	Images, local patterns	Conv + pooling	Weaker on long-range / global context
RNN / LSTM	Short-medium sequences	Recurrent hidden state	Sequential; fades on long context
Transformer	Text, vision, multimodal	Self-attention	$O(n^2)$ attention; data/compute hungry

Show hyperparameter starting points

14. Best practices & common pitfalls

✓ **Do**

Overfit a tiny batch first (loss → ~0) to prove the model + loss + data pipeline work. Split data before any preprocessing to avoid leakage. Track train and val curves together. Set seeds for reproducibility. Checkpoint the best model. Normalize inputs.

✦ **Debugging order**

If training won't learn: (1) check data/labels and shapes, (2) verify loss decreases on one batch, (3) lower the learning rate, (4) check for `zero_grad` and `train()` / `eval()` mode, (5) inspect gradient norms.

✕ **Data leakage**

Fitting a scaler/encoder on the full dataset (including validation/test) inflates scores and fails in production. Fit transforms on train only, then apply to val/test.

⚠ **When deep learning is NOT the answer**

On small tabular datasets, gradient-boosted trees (XGBoost, LightGBM) usually win with less tuning. If you have little data, no GPU budget, need full interpretability, or a simple baseline already meets the bar — don't reach for a deep net. Deep learning shines with large, high-dimensional, unstructured data (images, text, audio) and ample compute.

15. Interview Questions & Answers

Why do neural networks need nonlinear activation functions?

Without them, a stack of linear layers composes to a single linear transformation, so the network can only model linear relationships regardless of depth. Nonlinearities let the model approximate complex, non-linear functions and curved decision boundaries.

Explain backpropagation in one paragraph.

It is the chain rule applied efficiently over the computation graph. The forward pass caches each layer's activations. The backward pass starts from the loss and propagates the gradient of the loss with respect to each intermediate value from output to input, reusing cached values so each gradient is computed once. The result is $\partial L/\partial \theta$ for every parameter, which the optimizer uses to update weights.

What causes vanishing/exploding gradients and how do you fix them?

Repeated multiplication of layer Jacobians in deep nets; if their magnitudes are consistently below one, gradients vanish; above one, they explode. Fixes: ReLU/GELU activations, residual (skip) connections, batch/layer normalization, proper initialization (He/Xavier), gradient clipping, and gating (LSTM/GRU) for sequences.

Batch size and learning rate — how are they related?

Larger batches give lower-variance gradient estimates, so you can often use a larger learning rate; a common heuristic is linear scaling (double the batch, roughly double the LR) with a warmup. Very large batches can hurt generalization, so a schedule and warmup matter.

Dropout vs. BatchNorm — what do they do and when are they active?

Dropout is regularization: it randomly zeroes activations during training to prevent co-adaptation, and is turned off at inference. BatchNorm normalizes activations using batch statistics to stabilize and accelerate training; at inference it uses running (population) statistics. Both behave differently in train vs. eval mode, which is why calling `model.eval()` matters.

Why did transformers replace RNNs for most sequence tasks?

RNNs are inherently sequential (step t depends on t−1), limiting GPU parallelism and still losing long-range information. Self-attention relates any two positions directly with a constant number of sequential operations and parallelizes across the sequence, so it trains far faster on large data and captures long-range dependencies better.

What is the role of the √d_k scaling in attention?

Dot products of high-dimensional query/key vectors grow with dimension, pushing softmax into saturated regions with tiny gradients. Dividing by √d_k keeps the variance of the scores roughly constant so the softmax stays in a well-conditioned range.

How do you tell overfitting from underfitting, and what do you do about each?

Underfitting: both training and validation loss are high → increase capacity, train longer, reduce regularization, improve features. Overfitting: training loss is low but validation loss is high and rising → add regularization (dropout, weight decay, augmentation), get more data, or reduce capacity / use early stopping.

What is transfer learning and when do you freeze vs. fine-tune?

Reusing a model pretrained on a large corpus for a new task. With little data or a similar domain, freeze the backbone and train only a new head. With more data or a distant domain, unfreeze upper layers and fine-tune with a small learning rate. For LLMs, parameter-efficient methods like LoRA adapt the model cheaply.

Why use cross-entropy instead of MSE for classification?

Cross-entropy pairs with softmax to produce well-calibrated probabilities and gives strong, non-saturating gradients when predictions are confidently wrong, speeding learning. MSE on softmax outputs yields small gradients in saturated regions and treats the problem as regression, which converges slower and less reliably.

When would you NOT use deep learning?

> Small or purely tabular datasets (gradient-boosted trees often win), strict interpretability or latency/compute constraints, or when a simple model already meets requirements. Deep learning pays off with large, unstructured, high-dimensional data and sufficient compute.

> **What does a residual (skip) connection do?**
>
> It adds a layer's input to its output (`y = x + F(x)`), creating a gradient "highway" so signals bypass layers. This mitigates vanishing gradients and lets networks go very deep (ResNets, transformers) by letting each block learn a residual correction rather than a full transformation.

16. Further learning

- Build intuition: Andrej Karpathy's "Neural Networks: Zero to Hero" (micrograd → GPT from scratch).
- Foundations: the *Deep Learning* book (Goodfellow, Bengio, Courville) and the *Dive into Deep Learning* (d2l.ai) interactive text.
- Papers to read in order: "ImageNet Classification with Deep CNNs" (AlexNet), "Deep Residual Learning" (ResNet), "Attention Is All You Need" (transformer), "BERT", and "LoRA".
- Practice: reproduce a CNN on CIFAR-10 and a small character-level transformer; then fine-tune a pretrained model on your own dataset.
- Production next steps: mixed-precision training, learning-rate schedules, distributed/data-parallel training, quantization, and serving — see the MLOps guide.